Cadastro e Listagem de Fornecedores
React + Vite e Spring Boot

1 Objetivo e visão geral

Este roteiro implementa duas páginas simples no frontend React:

- cadastro de fornecedores;
- listagem de fornecedores cadastrados.

A página inicial apresenta somente dois links para acessar essas páginas. O backend Spring Boot disponibiliza uma API REST para consultar e inserir fornecedores no PostgreSQL.

A configuração Vite informada já encaminha chamadas iniciadas por `/api` para `http://127.0.0.1:808` Portanto, o React deve chamar caminhos como `/api/fornecedores`, sem escrever o endereço completo do backend no código frontend.

Importante: os exemplos abaixo usam o pacote Java `com.example.demo`. Substitua esse pacote pelo pacote-base real da sua aplicação Spring Boot. O backend deve estar configurado para executar na porta 8088.

2 1. Conferir as dependências do Spring Boot

No Spring Initializr, confirme que o projeto possui estas dependências:

- Spring Web;
- Spring Data JPA;
- PostgreSQL Driver.

Se o projeto já foi criado sem alguma delas, adicione-a ao `pom.xml` e atualize o projeto Maven no VS Code.

3 2. Configurar a conexão com o PostgreSQL

No arquivo `src/main/resources/application.properties`, configure a conexão com o banco. Ajuste o nome do banco, usuário e senha para os valores do seu ambiente.

```
server.port=8088

spring.datasource.url=jdbc:postgresql://localhost:5432/NOME_DO_BANCO
spring.datasource.username=SEU_USUARIO
spring.datasource.password=SUA_SENHA
spring.datasource.driver-class-name=org.postgresql.Driver
```

```
spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.default_schema=controlemerc
```

A tabela será administrada pelo SQL, por isso `ddl-auto=none` evita que o Hibernate tente criá-la ou alterá-la. O schema padrão é `controlemerc`.

A definição PostgreSQL da coluna de data deve usar `DEFAULT now()`:

```
CREATE TABLE IF NOT EXISTS controlemerc.fornecedores (
    id_fornecedor SERIAL PRIMARY KEY,
    nome_fantasia VARCHAR(100),
    razao_social VARCHAR(150),
    cnpj VARCHAR(18),
    telefone VARCHAR(15),
    email VARCHAR(100),
    cidade VARCHAR(50),
    estado CHAR(2),
    data_cadastro TIMESTAMP DEFAULT now()
);
```

Se a tabela já existe, não é necessário executá-la novamente apenas por causa deste roteiro.

4 3. Criar a entidade Fornecedor

Crie o arquivo `src/main/java/com/example/demo/entity/Fornecedor.java`. A entidade mapeia as colunas da tabela. O identificador é gerado pelo banco e `dataCadastro` está marcado como não inserível nem atualizável pelo JPA.

```
package com.example.demo.entity;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "fornecedores", schema = "controlemerc")
public class Fornecedor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_fornecedor")
    private Integer idFornecedor;

    @Column(name = "nome_fantasia", length = 100)
    private String nomeFantasia;

    @Column(name = "razao_social", length = 150)
    private String razaoSocial;

    @Column(name = "cnpj", length = 18)
    private String cnpj;

    @Column(name = "telefone", length = 15)
```

```
25      private String telefone;
26
27      @Column(name = "email", length = 100)
28      private String email;
29
30      @Column(name = "cidade", length = 50)
31      private String cidade;
32
33      @Column(name = "estado", length = 2)
34      private String estado;
35
36      @Column(name = "data_cadastro",
37              insertable = false, updatable = false)
38      private LocalDateTime dataCadastro;
39
40      public Fornecedor() {
41      }
42
43      public Integer getIdFornecedor() {
44          return idFornecedor;
45      }
46
47      public void setIdFornecedor(Integer idFornecedor) {
48          this.idFornecedor = idFornecedor;
49      }
50
51      public String getNomeFantasia() {
52          return nomeFantasia;
53      }
54
55      public void setNomeFantasia(String nomeFantasia) {
56          this.nomeFantasia = nomeFantasia;
57      }
58
59      public String getRazaoSocial() {
60          return razaoSocial;
61      }
62
63      public void setRazaoSocial(String razaoSocial) {
64          this.razaoSocial = razaoSocial;
65      }
66
67      public String getCnpj() {
68          return cnpj;
69      }
70
71      public void setCnpj(String cnpj) {
72          this.cnpj = cnpj;
73      }
74
75      public String getTelefone() {
76          return telefone;
77      }
```

```
78
79      public void setTelefone(String telefone) {
80          this.telefone = telefone;
81      }
82
83      public String getEmail() {
84          return email;
85      }
86
87      public void setEmail(String email) {
88          this.email = email;
89      }
90
91      public String getCidade() {
92          return cidade;
93      }
94
95      public void setCidade(String cidade) {
96          this.cidade = cidade;
97      }
98
99      public String getEstado() {
100         return estado;
101     }
102
103     public void setEstado(String estado) {
104         this.estado = estado;
105     }
106
107     public LocalDateTime getDataCadastro() {
108         return dataCadastro;
109     }
110 }
```

Não é necessário criar um setter para `dataCadastro`. O frontend também não enviará `idFornecedor` nem `dataCadastro` no cadastro.

5 4. Criar o repository

Crie `src/main/java/com/example/demo/repository/FornecedorRepository.java`:

```
1 package com.example.demo.repository;
2
3 import com.example.demo.entity.Fornecedor;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface FornecedorRepository
7         extends JpaRepository<Fornecedor, Integer> {
8 }
```

O `JpaRepository` já fornece o método `findAll()` para listar e `save()` para inserir ou atualizar. Neste roteiro, a API usa `save()` apenas para inserir um objeto novo, sem identificador.

6 5. Criar o service

Crie src/main/java/com/example/demo/service/FornecedorService.java:

```
package com.example.demo.service;

import com.example.demo.entity.Fornecedor;
import com.example.demo.repository.FornecedorRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class FornecedorService {

    private final FornecedorRepository repository;

    public FornecedorService(FornecedorRepository repository) {
        this.repository = repository;
    }

    public List<Fornecedor> listar() {
        return repository.findAll();
    }

    public Fornecedor inserir(Fornecedor fornecedor) {
        // N o aceitar identificador enviado pelo cliente.
        fornecedor.setIdFornecedor(null);
        return repository.save(fornecedor);
    }
}
```

7 6. Criar o controller REST

Crie src/main/java/com/example/demo/controller/FornecedorController.java:

```
package com.example.demo.controller;

import com.example.demo.entity.Fornecedor;
import com.example.demo.service.FornecedorService;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/fornecedores")
public class FornecedorController {

    private final FornecedorService service;

    public FornecedorController(FornecedorService service) {
        this.service = service;
    }
```

```
18
19      @GetMapping
20      public List<Fornecedor> listar() {
21          return service.listar();
22      }
23
24      @PostMapping
25      @ResponseStatus(HttpStatus.CREATED)
26      public Fornecedor inserir(@RequestBody Fornecedor fornecedor) {
27          // A API nunca usa um ID recebido no corpo da requisi  o.
28          fornecedor.setIdFornecedor(null);
29          return service.inserir(fornecedor);
30      }
31  }
```

A API disponibiliza:

- `GET /api/fornecedores`: retorna a lista em JSON;
- `POST /api/fornecedores`: insere um fornecedor enviado em JSON.

A coluna `id`$_{f}ornecedornoenviadanoINSERTpeloJPAquandooobjetonovotemidentificadornulo.Aco$

8 7. Criar as páginas React

Na pasta src, crie a pasta pages. A estrutura ficará semelhante a esta:

```
src/
  App.jsx
  main.jsx
  pages/
    Home.jsx
    FornecedorCadastro.jsx
    FornecedorLista.jsx
```

Não é obrigatório instalar React Router para este exemplo simples. O componente App.jsx controla qual página aparece. O arquivo main.jsx continua sendo o ponto de entrada que monta o React na página HTML.

8.1 7.1. Página inicial: Home.jsx

Crie src/pages/Home.jsx:

```
export default function Home({ abrirCadastro, abrirLista }) {
  return (
    <main>
      <h1>Fornecedores</h1>
      <ul>
        <li>
          <a href="#" onClick={(e) => {
            e.preventDefault();
            abrirLista();
          }}>
            Listar fornecedores
```

```
          </a>
        </li>
        <li>
          <a href="#" onClick={(e) => {
            e.preventDefault();
            abrirCadastro();
          }}>
            Cadastrar fornecedor
          </a>
        </li>
      </ul>
    </main>
  );
}
```

8.2 7.2. Página de cadastro: FornecedorCadastro.jsx

Crie src/pages/FornecedorCadastro.jsx:

```
import { useState } from 'react';

const vazio = {
  nomeFantasia: '',
  razaoSocial: '',
  cnpj: '',
  telefone: '',
  email: '',
  cidade: '',
  estado: ''
};

export default function FornecedorCadastro({ voltar }) {
  const [fornecedor, setFornecedor] = useState(vazio);
  const [mensagem, setMensagem] = useState('');

  function alterar(event) {
    const { name, value } = event.target;
    setFornecedor({ ...fornecedor, [name]: value });
  }

  async function enviar(event) {
    event.preventDefault();
    setMensagem('');

    try {
      const resposta = await fetch('/api/fornecedores', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(fornecedor)
      });

      if (!resposta.ok) {
```

```
34          throw new Error('Falha ao cadastrar fornecedor.');
35        }
36
37        setMensagem('Fornecedor cadastrado com sucesso.');
38        setFornecedor(vazio);
39      } catch (erro) {
40        setMensagem('Erro ao cadastrar. Verifique o backend e o banco
      .');
41      }
42    }
43
44    return (
45      <main>
46        <h1>Cadastro de fornecedor</h1>
47        <form onSubmit={enviar}>
48          <p>
49            <label>Nome fantasia<br />
50              <input name="nomeFantasia" maxLength="100"
51                value={fornecedor.nomeFantasia} onChange={alterar} />
52            </label>
53          </p>
54          <p>
55            <label>Raz o social<br />
56              <input name="razaoSocial" maxLength="150"
57                value={fornecedor.razaoSocial} onChange={alterar} />
58            </label>
59          </p>
60          <p>
61            <label>CNPJ<br />
62              <input name="cnpj" maxLength="18"
63                value={fornecedor.cnpj} onChange={alterar} />
64            </label>
65          </p>
66          <p>
67            <label>Telefone<br />
68              <input name="telefone" maxLength="15"
69                value={fornecedor.telefone} onChange={alterar} />
70            </label>
71          </p>
72          <p>
73            <label>E-mail<br />
74              <input name="email" type="email" maxLength="100"
75                value={fornecedor.email} onChange={alterar} />
76            </label>
77          </p>
78          <p>
79            <label>Cidade<br />
80              <input name="cidade" maxLength="50"
81                value={fornecedor.cidade} onChange={alterar} />
82            </label>
83          </p>
84          <p>
85            <label>Estado (UF)<br />
```

```
            <input name="estado" maxLength="2"
              value={fornecedor.estado} onChange={alterar} />
          </label>
        </p>
        <button type="submit">Salvar</button>
      </form>

      <p>{mensagem}</p>
      <p><a href="#" onClick={(e) => {
        e.preventDefault();
        voltar();
      }}>Voltar ao in cio </a></p>
    </main>
  );
}
```

Observe que o objeto enviado pelo formulário contém somente os sete campos de negócio. Não há idFornecedor nem dataCadastro no JSON enviado ao backend.

8.3 7.3. Página de listagem: FornecedorLista.jsx

Crie src/pages/FornecedorLista.jsx:

```
import { useEffect, useState } from 'react';

export default function FornecedorLista({ voltar }) {
  const [fornecedores, setFornecedores] = useState([]);
  const [mensagem, setMensagem] = useState('Carregando...');

  useEffect(() => {
    fetch('/api/fornecedores')
      .then((resposta) => {
        if (!resposta.ok) {
          throw new Error('Falha ao consultar fornecedores.');
        }
        return resposta.json();
      })
      .then((dados) => {
        setFornecedores(dados);
        setMensagem(dados.length === 0
          ? 'Nenhum fornecedor cadastrado.'
          : '');
      })
      .catch(() => {
        setMensagem('Erro ao carregar. Verifique o backend e o banco
   .');
      });
  }, []);

  return (
    <main>
      <h1>Fornecedores cadastrados</h1>
      <p>{mensagem}</p>
```

```
      {fornecedores.length > 0 && (
        <table border="1" cellPadding="5">
          <thead>
            <tr>
              <th>ID</th>
              <th>Nome fantasia</th>
              <th>Raz o social</th>
              <th>CNPJ</th>
              <th>Telefone</th>
              <th>E-mail</th>
              <th>Cidade</th>
              <th>Estado</th>
            </tr>
          </thead>
          <tbody>
            {fornecedores.map((f) => (
              <tr key={f.idFornecedor}>
                <td>{f.idFornecedor}</td>
                <td>{f.nomeFantasia}</td>
                <td>{f.razaoSocial}</td>
                <td>{f.cnpj}</td>
                <td>{f.telefone}</td>
                <td>{f.email}</td>
                <td>{f.cidade}</td>
                <td>{f.estado}</td>
              </tr>
            ))}
          </tbody>
        </table>
      )}
      <p><a href="#" onClick={(e) => {
        e.preventDefault();
        voltar();
      }}>Voltar ao in cio </a></p>
    </main>
  );
}
```

8.4 7.4. Controlar as páginas em App.jsx

Substitua o conteúdo de src/App.jsx por:

```
import { useState } from 'react';
import Home from './pages/Home';
import FornecedorCadastro from './pages/FornecedorCadastro';
import FornecedorLista from './pages/FornecedorLista';

export default function App() {
  const [pagina, setPagina] = useState('home');

  if (pagina === 'cadastro') {
    return <FornecedorCadastro voltar={() => setPagina('home')} />;
```

```
  }

  if (pagina === 'lista') {
    return <FornecedorLista voltar={() => setPagina('home')} />;
  }

  return (
    <Home
      abrirCadastro={() => setPagina('cadastro')}
      abrirLista={() => setPagina('lista')}
    />
  );
}
```

8.5 7.5. Conferir main.jsx

Normalmente, o Vite já cria esse arquivo. Confira se ele importa e renderiza App. Em geral, o conteúdo é:

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Se o arquivo existente já faz essa montagem, não precisa alterá-lo.

9 8. Executar a aplicação

Abra dois terminais no VS Code, um para cada parte.

1. No terminal do backend, na pasta que contém o pom.xml, execute:

```
./mvnw spring-boot:run
```

   No PowerShell do Windows, use .\mvnw.cmd spring-boot:run se o comando com barra não funcionar.

2. No terminal do frontend, na pasta que contém o package.json, execute:

```
npm run dev
```

3. Abra no navegador o endereço informado pelo Vite, normalmente http://localhost:517

4. Clique em ‘‘Listar fornecedores’’ para consultar os dados ou em ‘‘Cadastrar fornecedor’’ para inserir um registro.

Os dois processos precisam ficar executando simultaneamente, cada um em seu terminal. O backend deve estar disponível na porta 8088, conforme o proxy configurado.

10 9. Testes e problemas comuns

- Erro de conexão no cadastro ou na listagem: confirme que Spring Boot está em execução na porta 8088 e que o PostgreSQL está acessível.
- Erro 404: confirme que o controller usa @RequestMapping("/api/fornecedores") e que o frontend chama /api/fornecedores.
- Erro de tabela ou schema: confirme que a tabela existe em controlemerc.fornecedor e que o usuário do banco tem permissão para consultá-la e inserir registros.
- Erro de dependência Java: confirme que Spring Web, Spring Data JPA e PostgreSQL Driver estão presentes no projeto.
- Campos vazios: os campos da tabela aceitam valores nulos conforme o SQL apresentado. Se desejar torná-los obrigatórios, faça a validação no frontend e no backend.

11 Resumo do fluxo

1. O usuário abre uma das duas opções na página inicial React.
2. O cadastro envia um JSON com os campos do fornecedor para POST /api/fornecedores.
3. O controller chama o service, que usa o repository para persistir a entidade.
4. O PostgreSQL gera id_fornecedor e data_cadastro.
5. A listagem chama GET /api/fornecedores, e o React exibe os registros retornados.